HOW TO BECOME AN AI/ML ENGINEER AND GET A JOB AT BIGTECH

Eldan Nomad
Ex-Sr. AI/ML Engineer @ Apple & Microsoft
Founder, E-Nomad | @eldan.nomad

WHAT'S INSIDE THIS GUIDE

~3–4 months	~9–14 months	15–20 hrs/week	7 Phases
Math to LLMs	Foundations to Offer	Weekly Commitment	Structured Path

Comment **AIML** on any post @ **@eldan.nomad** to receive this full PDF guide.

ABOUT THE AUTHOR

This guide was built from real experience inside BigTech — not theory. Ansagan Yelubayev spent years as a Senior AI/ML Engineer at and , shipping production ML systems at scale. He has been through every layer of FAANG hiring — the coding loops, ML design sessions, and behavioral panels — and now runs **E-Nomad**, a digital AI engineering practice, while teaching thousands of developers through **@eldan.nomad**.

Apple	Sr. AI/ML Engineer — Production LLM & multimodal systems
Microsoft	Sr. AI/ML Engineer — Large-scale NLP infrastructure
E-Nomad	Founder — AI engineering for global clients
@eldan.nomad	AI/Dev educator — Guides, courses, content

"I built this guide to be the resource I wish existed when I was preparing — precise, battle-tested, and focused on what actually moves the needle at FAANG."

BIGTECH HIRING REALITY

Most candidates fail BigTech ML loops not because they lack ML knowledge — but because they treat it as a pure ML interview. FAANG hiring for ML roles evaluates four distinct pillars simultaneously. Weakness in any single one ends the offer.

01 **Coding (DSA)**
LeetCode-style algo problems in every loop. Non-negotiable.

02 **ML Breadth**
Depth in theory: math, models, architectures, training methods.

03 **ML System Design**
Design end-to-end ML systems at scale. The most differentiating round.

04 **Behavioral**
Leadership principles, project ownership, ambiguity, conflict.

The 7-Phase Roadmap Overview

Phase 1	Math & Programming Foundations	3–4 wks
Phase 2	Classical Machine Learning	2–3 wks
Phase 3	Deep Learning & PyTorch	3–4 wks
Phase 4	Transformers & Modern LLMs	2–3 wks
Phase 5	Engineering & Systems	Ongoing
Phase 6	Interview Preparation (FAANG)	3–4 mo
Phase 7	Application Strategy	1–2 mo

01

Mathematical & Programming Foundations

3–4 Weeks | 10–15 hrs/week

This is the bedrock. Every model, every architecture, every optimization algorithm you will ever work with is built on these concepts. Do not rush this phase.

Mathematics

- Linear Algebra — vectors, matrices, dot products, eigendecomposition, SVD. These map directly to weight matrices, attention heads, and dimensionality reduction.
- Multivariable Calculus — partial derivatives, Jacobians, the chain rule. The chain rule IS backpropagation. Internalize it.
- Probability & Statistics — distributions, Bayes' theorem, expectation, variance, KL divergence, MLE. Required for every probabilistic model.
- Optimization — gradient descent (SGD, momentum, Adam), learning rate schedules, convexity, saddle points, local minima.

Programming

- Professional Python — list comprehensions, generators, decorators, context managers, type hints, dataclasses.
- NumPy & Pandas — vectorized operations, broadcasting, DataFrame manipulation. Understand why loops are slow and matrices are fast.
- Git workflow — branching, rebasing, PR discipline, code reviews.
- Development hygiene — virtual environments, modular code structure, unit testing with pytest, debugging.

Recommended Resources

→ 3Blue1Brown: Essence of Linear Algebra (YouTube — free)

→ Mathematics for Machine Learning — Deisenroth, Faisal, Ong (PDF free online)

→ Fluent Python — Ramalho (O'Reilly)

Key Insight: Spend at least 2 weeks here even if you feel comfortable. Gaps in math surface painfully in FAANG ML Breadth rounds.

02

Classical Machine Learning

2–3 Weeks

Before building LLMs, you must own the fundamentals that all modern ML builds on. FAANG ML Breadth rounds routinely test these concepts in depth.

Core Algorithms

- Supervised: Linear & Logistic Regression (understand the math, not just the API), Decision Trees, Random Forests, Gradient Boosting (XGBoost, LightGBM).
- Unsupervised: k-Means, DBSCAN, PCA, t-SNE, autoencoders.
- SVMs — the kernel trick, margin maximization, dual formulation.
- Ensemble methods — bagging vs. boosting, why ensembles reduce variance.

ML Engineering Concepts

- Bias-variance trade-off — the single most important concept in applied ML.
- Regularization — L1 (Lasso, sparse features), L2 (Ridge, weight decay), dropout.
- Cross-validation — k-fold, stratified, leave-one-out.
- Evaluation metrics — precision, recall, F1, ROC-AUC, PR curves, NDCG. Know when to use each and why accuracy alone is misleading.
- Feature engineering — encoding, scaling, imputation, feature selection.
- Data leakage — how it happens and how to prevent it in pipelines.

Recommended Resources

→ Hands-On Machine Learning (3rd Ed.) — Aurélien Géron (O'Reilly)

→ Andrew Ng's Machine Learning Specialization (Coursera)

→ scikit-learn documentation — work through at least 3 real datasets end-to-end

Key Insight: Build a complete ML pipeline from scratch on a Kaggle dataset. Document your decisions. This becomes portfolio content.

03

Deep Learning & PyTorch

3–4 Weeks

Deep learning is the core of modern AI/ML engineering. PyTorch is the dominant framework at every major AI lab. Master it, not just know it.

Deep Learning Fundamentals

- Neural network basics — forward pass, loss computation, backward pass (autograd).
- Activation functions — sigmoid, tanh, ReLU, GeLU, SiLU. Why ReLU over sigmoid? Vanishing gradients. Be ready to explain this in an interview.
- Loss functions — cross-entropy, MSE, focal loss, contrastive loss. How do you choose? Match loss to task.
- Optimizers — SGD with momentum, Adam, AdamW, learning rate warmup, cosine annealing. Understand the math behind Adam's adaptive moments.
- Regularization — dropout (at training vs inference), batch norm, layer norm, weight decay. Know the difference between batch and layer norm.
- Architecture patterns — CNNs (convolution, pooling, receptive field), RNNs/LSTMs (vanishing gradient problem, cell state), Residual connections (why ResNet worked).

PyTorch Mastery

- Core API — tensor operations, autograd, nn.Module, custom layers.
- Training loop — DataLoader, Dataset, optimizer step, gradient clipping, mixed precision (torch.cuda.amp).
- GPU operations — .to(device), memory management, avoiding CPU-GPU bottlenecks.
- Checkpointing — saving/loading state_dict, resuming training.
- HuggingFace Transformers — from_pretrained, tokenizers, Trainer API, AutoModel, AutoTokenizer.
- HuggingFace Datasets — streaming, map, filter, DataCollator.

Recommended Resources

→ Deep Learning Specialization — Andrew Ng (Coursera)

→ Neural Networks: Zero to Hero — Andrej Karpathy (YouTube, free)

→ PyTorch official tutorials — pytorch.org/tutorials

Key Insight: Karpathy's GPT-from-scratch video is mandatory. It directly maps to understanding LLM internals required in Phase 4 and in interviews.

04

Transformers & Modern LLMs

2–3 Weeks

This is your primary technical differentiation as an AI/ML engineer in 2024–25. Go beyond surface-level API usage — understand architectures at the weight level.

Transformer Architecture

- Attention mechanism — query, key, value matrices. Why attention instead of RNNs? Parallelism, global context. Understand the O(n^2) complexity.
- Multi-head attention — why multiple heads? Different attention patterns.
- Positional encodings — absolute (sinusoidal), relative, RoPE, ALiBi.
- Encoder-only (BERT), decoder-only (GPT), encoder-decoder (T5, Bart). When to use which?
- Layer norm placement — Pre-LN vs Post-LN. Why Pre-LN stabilizes training.
- KV cache — what it is, why it matters for inference speed.

Modern LLM Techniques

- Tokenization — BPE (Byte-Pair Encoding), SentencePiece, tiktoken. Implement BPE from scratch at least once.
- Fine-tuning — full fine-tuning vs parameter-efficient (PEFT).
- LoRA (Low-Rank Adaptation) — the math: W' = W + AB where rank(AB) << rank(W). Why this works and what it freezes. QLoRA for memory efficiency.
- SFT (Supervised Fine-Tuning) vs RLHF (Reinforcement Learning from Human Feedback). PPO, reward modeling, preference data.
- Quantization — INT8, INT4, GPTQ, bitsandbytes. Trade-offs.
- Inference optimization — vLLM, continuous batching, PagedAttention, speculative decoding.

Recommended Resources

→ Attention Is All You Need — Vaswani et al. (arXiv:1706.03762)

→ LLaMA 2 paper — Touvron et al. (arXiv:2307.09288)

→ PEFT library docs — huggingface.co/docs/peft

→ Sebastian Raschka's LLM series (Magazine-level quality, free online)

Key Insight: Practice fine-tuning Llama 3.2 1B or Qwen2.5 0.5B on a custom dataset. Getting hands dirty with LoRA before the course covers it gives you depth.

05

Engineering & Systems Skills

Ongoing — Build in Parallel

FAANG ML roles are engineering roles. The ability to ship, monitor, and maintain ML systems at scale is what separates a research background from a production hire.

ML Infrastructure

- Experiment tracking — Weights & Biases (wandb), MLflow. Log everything: hyperparams, metrics, artifacts, model versions.
- Model serving — FastAPI for REST endpoints, vLLM for LLM serving, Ollama for local inference. Understand latency vs throughput trade-offs.
- Vector databases — pgvector (Postgres), Qdrant, Pinecone, Weaviate. Understand HNSW indexing, cosine vs dot product similarity.
- Docker — containerize every project. Write clean Dockerfiles.
- Cloud basics — S3/GCS for data, EC2/GCP for training, SageMaker/Vertex AI.

RAG & Agent Systems

- RAG pipeline — chunking strategies, embedding models (text-embedding-3, BGE), retrieval (dense, sparse, hybrid), reranking, generation.
- Evaluation — RAGAS framework, faithfulness, answer relevancy, context precision.
- Agent frameworks — LangChain, LlamaIndex, DSPy. Use as tools, not magic.
- Tool calling — OpenAI function calling, Anthropic tools API format.
- Prompt versioning — version your prompts like code. Track changes, test regressions.

Recommended Resources

→ Full Stack LLM Bootcamp — fullstackdeeplearning.com (free)

→ Designing ML Systems — Chip Huyen (O'Reilly)

→ vLLM docs — vllm.readthedocs.io

Key Insight: Every project you build should be containerized, tracked with W&B;, and deployed — even locally. This is your portfolio.

06

Interview Preparation — FAANG Loop

3–4 Months, Parallel to Phase 3–5

This is the phase most engineers underestimate. Technical preparation alone is insufficient. FAANG loops are structured, high-pressure, and test multiple skills simultaneously. You must train for the format, not just the content.

Every FAANG ML loop includes 1–2 LeetCode-style coding rounds. These are not optional. You cannot compensate with strong ML knowledge.

Data Structures to Master

- Arrays & Strings — two pointers, sliding window, prefix sums
- Hash Maps & Sets — frequency counts, anagram detection, two-sum family
- Linked Lists — fast/slow pointer, reversal, cycle detection
- Stacks & Queues — monotonic stack, BFS with queue, next greater element
- Trees & BSTs — DFS (pre/in/post order), BFS, lowest common ancestor
- Tries — prefix trees, autocomplete, word search
- Heaps — top-K problems, merge K sorted lists, median finder
- Graphs — BFS/DFS, connected components, topological sort, union-find

Algorithm Patterns to Own

Two Pointers	Pair sum, container with most water, palindrome
Sliding Window	Max sum subarray, longest substring without repeat
Binary Search	Search in rotated array, find first/last position
BFS/DFS	Level-order traversal, shortest path, word ladder
Dynamic Programming	Climbing stairs → coin change → edit distance → knapsack
Backtracking	Subsets, permutations, N-queens, Sudoku solver
Greedy	Interval scheduling, jump game, task scheduler
Topological Sort	Course schedule, alien dictionary, build order

LeetCode Strategy

- Start with Blind 75 — the canonical list. Every problem tagged with a pattern.
- Progress to NeetCode 150 — covers all patterns with video explanations.
- Final 4 weeks — company-tagged problems for your target companies (Meta, Google, Amazon).
- Target: 250–300 problems total, but pattern mastery matters more than volume.

- Time yourself: 20 min for easy, 35 min for medium, 50 min for hard. Stop and review.
- Verbalize: narrate your thought process while solving. Silent solving fails live interviews.
- Mock interviews: minimum 5–10 sessions on Pramp or Interviewing.io before any real loop.

→ NeetCode.io — free structured roadmap with video solutions
→ Cracking the Coding Interview — McDowell (book)
→ Elements of Programming Interviews — Aziz, Lee, Prakash

Expect rapid conceptual questions. The interviewer is not looking for surface-level answers — they probe until they find the boundary of your understanding. Know where that boundary is and be honest about it.

Foundations

- Why does gradient descent work? What happens when learning rate is too high/too low?
- Derive cross-entropy loss. Why does log-softmax pair well with NLL loss?
- Explain bias-variance trade-off. Give a concrete example of each.
- What is the difference between L1 and L2 regularization? When do you use each?
- Why does batch normalization help? What's the problem it solves? Layer norm vs batch norm.

Deep Learning

- Explain vanishing gradients. How do ResNets solve it? How do LSTMs address it?
- Why ReLU instead of sigmoid? What is dying ReLU? How does LeakyReLU help?
- What is dropout and why does it work as regularization? Behavior at inference time.
- Explain the Adam optimizer. What do the first and second moment estimates represent?
- When would you use MSE vs cross-entropy loss?

Transformers & LLMs

- Explain self-attention from scratch. What are Q, K, V matrices?
- Why does attention have O(n^2) complexity? What are flash attention / linear attention approaches?
- What does LoRA do? Why does low-rank decomposition work for fine-tuning?
- Explain RLHF end-to-end: reward model, PPO, preference data.
- What is the KV cache? Why does it matter for inference?
- Difference between SFT and RLHF. When would you use each?

→ Machine Learning Interviews — Khang Pham
→ Deep Learning Interviews — Elad Kalami

This is the highest-leverage round and the most differentiating at senior levels. You are given an open-ended problem and 45–60 minutes to design an ML system end-to-end. There is no single correct answer — the interviewer is evaluating structured thinking, trade-off reasoning, and production awareness.

The 8-Step Framework

01. Clarify Requirements
Ask about scale (DAU, QPS, latency SLAs), scope (which features matter), and constraints (budget, team size, existing infra). Never skip this — interviewers deliberately leave requirements vague to test this instinct.

02. Define Metrics
Offline metrics: AUC-ROC, NDCG, precision@k, BLEU, ROUGE, perplexity. Online metrics: CTR, conversion, DAU retention, revenue lift. Always define both and explain the gap between them.

03. Frame the ML Problem
Supervised/unsupervised/self-supervised? Classification/regression/ranking/generation? Pointwise vs pairwise vs listwise ranking? Justify your framing.

04. Data Collection & Labeling
Volume needed, collection strategy, labeling approach (crowdsourcing, programmatic, model-assisted), handling class imbalance, data versioning, privacy concerns.

05. Feature Engineering
What signals matter? User features, item features, context features, interaction features. Embedding representations. Feature freshness and staleness.

06. Model Architecture
Always start with a simple baseline (logistic regression, BM25). Justify why you'd move to a more complex model. For LLM-based systems: prompt engineering → RAG → fine-tuning hierarchy.

07. Training Pipeline
Data pipeline, feature store, training infra, distributed training strategy, experiment tracking, model registry, deployment gating.

08. Serving, Monitoring & Iteration
Latency vs throughput trade-offs, caching strategy, A/B testing design, monitoring (data drift, model degradation, feature drift), retraining triggers, rollback strategy, shadow deployment.

Must-Practice System Designs

- **Recommendation System** — YouTube, Netflix, TikTok feed — two-tower model, ANN retrieval
- **Search & Ranking** — Google, Amazon product search — BM25 + neural reranking
- **Ad Click Prediction** — CTR prediction at scale — logistic regression → DLRM
- **RAG Chatbot** — Customer support, internal knowledge base — your course project
- **Content Moderation** — Hate speech, NSFW detection — multi-label classification

- **Fraud Detection** — Transaction fraud — imbalanced classification, real-time scoring
- **LLM Fine-tuning Pipeline** — Domain adaptation, instruction tuning at production scale
- **Multimodal Search** — Image + text retrieval — CLIP embeddings, cross-modal fusion

→ Machine Learning System Design Interview — Ali Aminian & Alex Xu
→ Designing Machine Learning Systems — Chip Huyen (O'Reilly)
→ Evidently AI ML System Design case studies — evidentlyai.com

Every company frames this differently — Amazon's Leadership Principles, Meta's behavioral signals, Google's Googleyness — but all probe the same core themes: ownership, ambiguity, conflict, and measurable impact. Your E-Nomad founder experience and BigTech tenure are genuinely strong assets here.

The STAR Framework

- **Situation** — Set the scene with minimal context. One to two sentences.
- **Task** — What was your specific responsibility? Not the team's — yours.
- **Action** — What did YOU do? Use 'I', not 'we'. Be specific about decisions.
- **Result** — Quantify the impact. Numbers, percentages, time saved, revenue, users.

8 Core Stories to Prepare

Prepare exactly 8 rich STAR stories, each flexible enough to answer 3–4 different prompts. Rotate details based on the question.

- **Story 1:** A technically ambiguous project where you made a key architecture decision
- **Story 2:** A time you disagreed with a manager or senior engineer — and how it resolved
- **Story 3:** A project you led from idea to production impact
- **Story 4:** A time you failed and what you did about it
- **Story 5:** A situation where you had to move fast with incomplete information
- **Story 6:** A time you mentored someone or elevated your team
- **Story 7:** The most complex technical problem you solved and why it was hard
- **Story 8:** A time you influenced cross-team alignment without authority

07

Application Strategy

1–2 Months

Technical readiness is necessary but not sufficient. How you apply, position yourself, and negotiate is equally important. Treat the job search as a product launch.

Resume for ML Roles

- One page for under 10 years of experience. Clean, ATS-parseable format.
- Every bullet follows: **Action verb** → **What you did** → **Result with numbers**. Example: 'Reduced inference latency 40% by implementing vLLM continuous batching for production LLM serving.'
- Skills section: Python, PyTorch, HuggingFace, LangChain, SQL, Docker, AWS/GCP. Do not list things you cannot be interviewed on.
- Projects: link to GitHub with working demos. No placeholder repos.
- Tailor each resume to the specific role. Keyword-match with the JD for ATS.

Target Company Strategy

Stretch	Google DeepMind, Meta AI, Anthropic, OpenAI, Apple ML
Realistic	Amazon (AWS AI), Microsoft (Azure AI), Nvidia, Salesforce AI
Strong	Databricks, Cohere, Mistral, HuggingFace, Scale AI, Snowflake ML

The Referral Rule

Cold applications to FAANG have conversion rates below 2%. Referrals increase interview rates to 30–50%. Spend 20% of your application time on LinkedIn connections, alumni networks, and developer communities. One genuine referral is worth 50 cold applies.

Salary Negotiation

- Never give a number first. 'I'm focused on finding the right fit — what's the budgeted range?'
- Use levels.fyi to know the exact band before any negotiation conversation.
- Competing offers are the strongest leverage. Time them to arrive within the same window.
- Always negotiate base, equity, signing bonus, and vesting schedule separately.
- A 10-minute negotiation conversation can add $30K–$100K to total comp. Do it.

COMPLETE TIMELINE

Month 1	Phase 1 + 2	Math foundations, Python, Classical ML
Month 2	Phase 3	Deep Learning, PyTorch, train your first model
Month 3	Phase 4	Transformers, LLMs, fine-tune open model with LoRA
Month 4	Phase 5	Engineering skills, RAG system, deploy to production
Month 5–6	Phase 6A	LeetCode daily: Blind 75 → NeetCode 150
Month 6–7	Phase 6B/C/D	ML Breadth review, System Design practice, Behavioral stories
Month 8	Mock Loops	Full mock loops, application prep, target list
Month 9+	Phase 7	Active applications, phone screens, onsites, offer negotiation

Ready to Start?

This guide is your complete roadmap. Every phase is deliberate. Compress what you already know, go deep where you are weak, and treat the interview preparation as a separate skill that must be trained — not studied.

Follow **@eldan.nomad** for weekly AI engineering content, system design breakdowns, and real FAANG interview insights.

Comment "AIML" on any post to receive this guide directly.